University of Colombo School of Computing

SCS 1304 - Problem Solving Strategies and Computation Approaches

Lab Sheet 07

Brute Force & Divide-and-Conquer

Answer the following questions and you are allowed to use non programmable calculators.

1. A logistics company is sorting delivery addresses collected from different cities. To improve the sorting process, the company uses the **Merge Sort** algorithm. The following array represents the delivery IDs:

   [45, 18, 72, 11, 39, 56, 8, 91, 27, 63, 14, 50]

   A. Fill in a trace table to show each step performed by the Merge Sort algorithm while sorting the given array. Include all intermediate steps during both the **splitting** and **merging** phases.

   B. Draw the complete **Split-Merge Tree** for the given array, clearly showing how the array is divided into smaller subarrays and how they are merged to produce the final sorted array.

   C. During the merge phase, calculate the total number of element comparisons made while sorting the given array. Show your calculations.

   D. Explain the time complexity of Merge Sort for the **best-case, average-case, and worst-case** scenarios. Briefly explain why Merge Sort maintains the same time complexity regardless of the initial order of the elements.

Refer to the following before attempting Questions 02 and 03

- ★ Kadane's Algorithm: Find Maximum Subarray Sum in an Array I Codecademy
- ★ Maximum Subarray Sum - Kadane's Algorithm - GeeksforGeeks
- ★ Maximum Subarray Sum using Divide and Conquer algorithm - GeeksforGeeks
- ★ Maximum sum sub-array

2. A retail company is analysing the daily profit and loss values recorded over several days to identify the most profitable continuous period. The profit/loss values are given below:

[250, -120, 340, -180, 520, -90, 160, -240, 430, -110]

A. Explain how the **Brute Force** approach can be used to find the maximum subarray sum for the given data. Describe the steps involved and discuss its time complexity.

B. Using the Brute Force method, calculate the maximum subarray sum. List all possible contiguous subarrays, calculate their sums, and identify the subarray with the highest total.

C. Determine the total number of subarrays that must be examined and estimate the number of comparisons performed during the execution of the algorithm.

D. Explain why the Brute Force approach becomes inefficient as the size of the dataset increases. State the time complexity of the algorithm and recommend a more efficient algorithm for solving the Maximum Subarray Problem. Briefly compare the time complexities of the two approaches.

3. In **Question 2**, the retail company used the **Brute Force** approach to identify the period with the highest total profit. To improve performance for larger datasets, the company now decides to solve the **same problem** using the **Divide and Conquer** approach. Consider the same profit/loss values:

[250, -120, 340, -180, 520, -90, 160, -240, 430, -110]

A. Explain how the **Divide and Conquer** algorithm can be used to solve the Maximum Subarray Problem. Describe how the array is recursively divided into smaller subarrays, how the maximum subarray is determined for the left half, right half, and crossing subarray, and how these results are combined to obtain the final solution.

B. Draw the complete recursion tree for the given array, clearly showing how the array is divided until each subarray contains a single element (base case).

C. Apply the Divide and Conquer algorithm to solve the Maximum Subarray Problem for the given array. Calculate the maximum subarray sum for the **left half, the right half, and the crossing subarray** at the **initial division of the array**. Use these results to determine the final maximum subarray sum.

D. Explain the time complexity of the Divide and Conquer algorithm used to solve the Maximum Subarray Problem. Briefly explain why it is more efficient than the Brute Force approach for large datasets.

E. Compare the **Brute Force** approach used in Question 2 with the **Divide and Conquer** approach in terms of:

a. The main idea of each algorithm.

b. Time complexity.

c. Efficiency for small and large datasets.

d. Advantages and limitations of each approach.